

Points and Lines

Point

A point represents an exact location in space. It has no length, width, or thickness (no dimensions).

A point is usually named with a capital letter, such as A, B, or P. It is represented by a small dot.

Example:

Point A

Point B

Line

A line is a straight path that extends infinitely in both directions.

It has only one dimension: length. A line has no endpoints.

It is named using two points on the line (e.g., Line AB) or a lowercase letter (e.g., line l).

It is represented by a straight line with arrows at both ends.

Properties of a Line

Extends forever in both directions.

Contains infinitely many points.

The shortest distance between two points is a straight line.

Example:

Line AB ($\leftrightarrow$ AB)

CHAPTER – 3

LINE DRAWING ALGORITHMS:

1. SIMPLE LINE DRAWING ALGORITHM:

The Cartesian slope intercept equation for a straight line is:

$$y = mx + b \Rightarrow b = y - mx,$$

Where m = slope of line, b = y-intercept form.

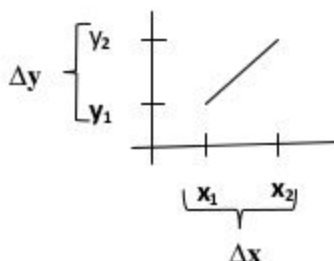
If 2 end points of a line segment is specified at position (x_1, y_1) and (x_2, y_2) . So the values for the slope ' m ' and y-intercept form ' b ' is:

$$m = \Delta y / \Delta x = (y_2 - y_1) / (x_2 - x_1)$$

$$\Rightarrow \Delta y = m \Delta x \text{ ----- (1) and}$$

$$\Delta x = \Delta y / m \text{ ----- (2)}$$

$$b = y_1 - mx_1$$



Here, ' Δy ' is the y-interval computed for the given x-interval Δx along a line is $\Delta y = m \Delta x$. Similarly the x-interval Δx corresponding to a specified Δy as: $\Delta x = \Delta y / m$.

For lines with slope magnitude, $|m| < 1$, ' Δx ' increases. So calculate ' Δy ' as: $\Delta y = m \Delta x$. For $|m| > 1$, ' Δy ' increases. So calculate ' Δx ' as: $\Delta x = \Delta y / m$. For $|m| = 1$, Δx and Δy both incremented and have the same value for horizontal and vertical segments.

Example: Draw a line for points (3, 2) to (7, 8).

Here, $(x_1, y_1) = (3, 2)$ and $(x_2, y_2) = (7, 8)$.

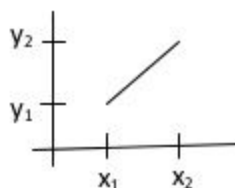
So $m = y_2 - y_1 / x_2 - x_1 = 8 - 2 / 7 - 3 = 6 / 4 = 3 / 2 > 1$. That is, for $m > 1$, Δy increases and $\Delta x = \Delta y / m$.

Δx	Δy	$x = x_1 + \Delta x$	$y = y_1 + \Delta y$
2/3	1	11/3	3
4/3	2	13/3	4
2	3	5	5
8/3	4	17/3	6
10/3	5	19/3	7
4	6	7 (x_2)	8 (y_2)

2. DIGITAL DIFFERENTIAL ANALYSER ALGORITHM (DDA):

This is a scan conversion line algorithm based on calculating either Δy or Δx as: $\Delta y = m\Delta x$ and $\Delta x = \Delta y / m$. We sample the line at unit intervals in one co-ordinate and determine corresponding integer values nearest the line path for the other co-ordinate.

Consider first a line with positive slope. If $m \leq 1$, we sample at unit x -intervals ($\Delta x = 1$) and calculate successive 'y' values as: $y_{k+1} = y_k + m$. (Δx increases by 1 always, so $\Delta x = 1$).



Where, k = subscript and integer values starting from 1, for 1st point and increase by 1 until the final end point is reached. Since 'm' can be any real number between 0 and

1, the calculated 'y' values must be rounded to the nearest integer.

If $m > 1$, we sample at unit y-intervals ($\Delta y = 1$) and calculate each succeeding 'x' values as: $x_{k+1} = x_k + 1/m$. (Δy increases by 1 always, so $\Delta y = 1$). Here, the lines are processes from left end point to right end point.

If the lines processed from right end to left end point, then $\Delta x = -1$ and $y_{k+1} = y_k - m$ (for, $m \leq 1$), when $\Delta y = -1$ and $m > 1$, then $x_{k+1} = x_k - 1/m$.

Example: Draw a line for points (3, 2) to (7, 8).

Here, $x_k = 3$, $y_k = 2$ and $m = y_2 - y_1 / x_2 - x_1 = 8 - 2 / 7 - 3 = 6/4 = 3/2 > 1$. So, $m > 1$, so y_k increases by 1 always.

$x_{k+1} = x_k + 1/m$	$y_{k+1} = y_k + m$
11/3	3
13/3	4
15/3 = 5	5
17/3	6
19/3	7
7 (x_2)	8 (y_2)

3. BRESENHAM'S LINE DRAWING ALGORITHM:

An accurate and efficient raster line generating algorithm is developed by Bresenham. In this approach, at slope $m < 1$, pixel positions along a line path are then determined by sampling at unit x-intervals. Starting from left end point (x_0, y_0) of a given line, we step to each successive column (x -

position) and plot the pixel, whose scan-line 'y'-value is closest to the line 'p'.

y_{k+3}				
y_{k+2}	$y = mx + b$			
y_{k+1}				
y_k				
	x_k	x_{k+1}	x_{k+2}	x_{k+3}

For k^{th} step, we determine the pixel at (x_k, y_k) is to be displayed we need to decide which pixel to plot in column, x_{k+1} and the choice of pixel position are: (x_{k+1}, y_k) and (x_{k+1}, y_{k+1}) . At x_{k+1} , we label vertical pixel separations from the mathematical line path as 'd₁' and 'd₂'.

So at x_{k+1} , the 'y' co-ordinate is:

$$y = m(x_k + 1) + b \text{ ----- (1)}$$

$$d_1 = y - y_k = m(x_k + 1) + b - y_k$$

$$d_2 = (y_k + 1) - y = y_k + 1 - m(x_k + 1) - b$$

The difference between these 2 separations is:

$$d_1 - d_2 = m(x_k + 1) + b - y_k - (y_k + 1) + m(x_k + 1) + b$$

$$= 2m(x_k + 1) + 2b - y_k - (y_k + 1) = 2m(x_k + 1) + 2b - y_k - y_k - 1$$

$$\Rightarrow d_1 - d_2 = 2m(x_k + 1) + 2b - 2y_k - 1 \text{ ----- (2)}$$

The decision parameter 'P_k' for k^{th} step in the line algorithm is calculated with substitution of $m = \Delta y / \Delta x$.

$$\text{So, } P_k = \Delta x(d_1 - d_2)$$

$$= \Delta x(2m(x_k + 1) + 2b - 2y_k - 1)$$

$$\begin{aligned}
&= \Delta x(2\Delta y/\Delta x (x_k + 1) + 2b - 2y_k - 1) \\
&= 2\Delta y(x_k + 1) + 2b\Delta x - 2y_k \Delta x - \Delta x \\
&= 2\Delta yx_k + 2\Delta y + 2b\Delta x - 2y_k \Delta x - \Delta x \\
&= 2\Delta yx_k - 2\Delta xy_k + 2\Delta y + 2b\Delta x - \Delta x \\
&= 2\Delta yx_k - 2\Delta xy_k + 2\Delta y + \Delta x(2b - 1) \\
&= 2\Delta yx_k - 2\Delta xy_k + C
\end{aligned}$$

$$\Rightarrow P_k = 2\Delta yx_k - 2\Delta xy_k + C \text{ ----- (3)}$$

The sign of ' P_k ' is same as the sign of $d_1 - d_2$, since $\Delta x > 0$, $C = \text{constant}$ and value of $C = 2\Delta y + \Delta x(2b - 1)$, independent of pixel position. If the pixel at ' y_k ' is closer to the line path than the pixel at y_{k+1} (i.e. $d_1 < d_2$), then the decision parameter ' P_k ' is negative.

At step $k + 1$, the decision parameter is evaluated as:

$$P_{k+1} = 2\Delta y(x_k + 1) - 2\Delta x(y_k + 1) + C \text{ ----- (4)}$$

Now, subtract equation – (3) from equation – (4), we get:

$$P_{k+1} - P_k = 2\Delta y(x_{k+1} - x_k) - 2\Delta x(y_{k+1} - y_k), \text{ but } x_{k+1} = x_k + 1, \text{ so that}$$

$$P_{k+1} = P_k + 2\Delta y(x_{k+1} - x_k) - 2\Delta x(y_{k+1} - y_k)$$

$$\Rightarrow P_{k+1} = P_k + 2\Delta y - 2\Delta x(y_{k+1} - y_k) \text{ ----- (5)}$$

Where, $y_{k+1} - y_k$ is either 0 or 1, depending on the sign of parameter P_k .

The recursive calculation of decision parameters is performed at each integer x-position, starting at the left co-ordinate end point of the line. The 1st parameter 'P₀' is evaluated from equation – (3), at the starting pixel position (x₀, y₀) and with 'm' evaluated as $\Delta y/\Delta x$. So, $P_0 = 2\Delta y - \Delta x$ --- (6).

ALGORITHM:

STEP 1: Input the two line end points and store the left end point in (x₀, y₀)

STEP 2: Load the (x₀, y₀) or the initial point of the line into the frame buffer then plot the first point.

STEP 3: Calculate the constant Δx , Δy , $2\Delta y$, $2\Delta y - 2\Delta x$ and obtain the starting value for the decision parameter: $P_0 = 2\Delta y - \Delta x$

STEP 4: At each 'x_k' along the starting at k = 0, perform the following point.

If $P_k < 0$, plot (x_k + 1, y_k) and $P_{k+1} = P_k + 2\Delta y$

If $P_k > 0$, plot (x_k + 1, y_k + 1) and $P_{k+1} = P_k + 2\Delta y - 2\Delta x$.

STEP 5: Repeat STEP – 4, Δx times.

Example: To illustrate the algorithm, we digitize the line with end points (20, 10) and (30, 18)

Here the line has a slope:

$$m = y_2 - y_1/x_2 - x_1 = 18 - 10/30 - 20 = 8/10 = 0.8 < 1$$

$$\Delta x = 30 - 20 = 10 \text{ and } \Delta y = 18 - 10 = 8.$$

The initial decision parameter is: $P_0 = 2\Delta y - \Delta x = 2 \times 8 - 10 = 16 - 10 = 6$ and the increments for the calculating successive decision parameters are: $\Delta x = 10$, $\Delta y = 8$, $2\Delta y = 16$, $2\Delta y - 2\Delta x = 16 - 20 = -4$. At $k = 0$, we plot the initial point $(x_k, y_k) = (x_0, y_0) = (20, 10)$ and determine successive pixel positions along the line path from the decision parameter as:

k	P_k	$(x_k + 1, y_k + 1)$	Decision parameter calculation
0	6	(21, 11)	$P_0 = 2\Delta y - \Delta x = 2 \times 8 - 10 = 16 - 10 = 6$
1	2	(22, 12)	$P_{k+1} = P_k + 2\Delta y - 2\Delta x = 6 + 2 \times 8 - 2 \times 10 = 2$
2	-2	(23, 12)	$P_{k+1} = P_k + 2\Delta y - 2\Delta x = 2 + 2 \times 8 - 2 \times 10 = -2$
3	14	(24, 13)	$P_{k+1} = P_k + 2\Delta y = -2 + 2 \times 8 = 14$
4	10	(25, 14)	$P_{k+1} = P_k + 2\Delta y - 2\Delta x = 14 + 2 \times 8 - 2 \times 10 = 10$
5	6	(26, 15)	$P_{k+1} = P_k + 2\Delta y - 2\Delta x = 10 + 2 \times 8 - 2 \times 10 = 6$
6	2	(27, 16)	$P_{k+1} = P_k + 2\Delta y - 2\Delta x = 6 + 2 \times 8 - 2 \times 10 = 2$
7	-2	(28, 16)	$P_{k+1} = P_k + 2\Delta y - 2\Delta x = 2 + 2 \times 8 - 2 \times 10 = -2$
8	14	(29, 17)	$P_{k+1} = P_k + 2\Delta y = -2 + 2 \times 8 = 14$
9	10	(30, 18)	$P_{k+1} = P_k + 2\Delta y - 2\Delta x = 14 + 2 \times 8 - 2 \times 10 = 10$

It will continue upto 10 times, starting from 0 to 9, because $\Delta x = 10$.

CIRCLE GENERATING ALGORITHMS:

1. PROPERTIES OF THE CIRCLE:

A Circle is defined as the set of points that are all at given distance 'r' from a center position (x_c, y_c) . This distance relationship is expressed by the Cartesian co-ordinates as:

Properties of a Circle

1. Definition

- A circle is the set of all points that are at a fixed distance r (radius) from a fixed point called the **center** (x_c, y_c) .  Chapter_3

2. Standard Equation

$$(x - x_c)^2 + (y - y_c)^2 = r^2$$

This represents a circle with center (x_c, y_c) and radius r .  Chapter_3

3. Circle at the Origin

- If the center is at $(0, 0)$, the equation becomes:

$$x^2 + y^2 = r^2$$

 Chapter_3

4. Finding Points on the Circle

- For a given x -coordinate:

$$y = y_c \pm \sqrt{r^2 - (x - x_c)^2}$$

This is used to calculate points on the circle boundary.  Chapter_3

5. Parametric (Polar) Form

- A circle can also be represented as:

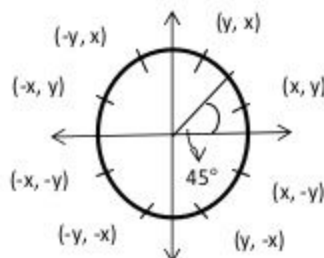
$$x = x_c + r \cos \theta, \quad y = y_c + r \sin \theta$$

where θ is the angle measured from the positive x -axis.  Chapter_3

6. Symmetry

- A circle is symmetric about:
 - the x -axis
 - the y -axis
 - the **origin**
 - the **45° line** (octant symmetry)
- Because of this **8-way symmetry**, only one octant needs to be computed; the remaining seven points can be obtained by reflection. This greatly reduces computation in circle drawing algorithms.

octants of the xy-plane. Taking the advantage of the circle symmetry, we can generate all pixel positions around a circle by calculating only the points within the sector from $x = 0$ to $x = y$.



2. MID-POINT CIRCLE/BRESENHAM'S CIRCLE GENERATING ALGORITHM:

We set up the algorithm to calculate pixel positions around a circle path centered at the co-ordinate origin $(0, 0)$. Then each calculated position (x, y) is moved to its proper screen position by adding ' x_c ' to ' x ' and ' y_c ' to ' y ', where (x_c, y_c) is the center position of the circle with radius ' r '.

Along the circle section from $x = 0$ to $x = y$ in 1st quadrant, the slope of the curve varies from 0 to 1. To apply the midpoint method, we define the circle function with $(0, 0)$ as its center is:

$$F_{\text{circle}}(x, y) = x^2 + y^2 - r^2 \quad \text{--- (1)}$$

It has 3 properties, at any point (x, y) . That is:

$$F_{\text{circle}}(x, y) = \begin{cases} < 0, & \text{if } (x, y) \text{ is inside the circle boundary} \\ = 0, & \text{if } (x, y) \text{ is on the circle boundary} \\ > 0, & \text{if } (x, y) \text{ is outside the circle boundary} \end{cases} \quad \text{--- (2)}$$

We have plotted the pixel at (x_k, y_k) and next to determine whether the pixel at position (x_{k+1}, y_k) or one at position (x_{k+1}, y_{k-1}) is closer to the circle. The decision parameter is the circle function and equation – (1) is evaluated at the midpoint between these 2 pixels (x_{k+1}, y_k) and (x_{k+1}, y_{k-1}) . So, the decision parameter is:

$$P_k = F_{\text{circle}}(x_{k+1}, y_{k-1/2}) = (x_{k+1})^2 + (y_{k-1/2})^2 - r^2 \dots (3)$$

$$[\text{Since midpoint} = (y_k + y_{k-1})/2 = (2y_k - 1)/2 = y_k - 1/2]$$

If $P_k < 0$, the midpoint is inside the circle and the pixel on scan line ' y_k ' is closer to the circle boundary, otherwise the midpoint is outside or on the circle boundary and we select the pixel on ' y_{k-1} '.

The recursive expression for the next decision parameter is obtain by evaluating the circle function at sampling position $x_{k+1} + 1 = x_k + 2$, then,

$$P_{k+1} = F_{\text{circle}}(x_{k+1} + 1, y_{k+1} - 1/2) = [(x_{k+1} + 1)]^2 + (y_{k+1} - 1/2)^2 - r^2$$

$$\Rightarrow P_{k+1} - P_k = 2(x_k + 1) + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) + 1$$

$$\Rightarrow P_{k+1} = P_k + 2(x_k + 1) + (y_{k+1}^2 - y_k^2) - (y_{k+1} - y_k) + 1 \dots (4),$$

where y_{k+1} is either ' y_k ' or ' y_{k-1} ', depending on the sign of P_k .

Increments for obtaining P_{k+1} are either $2x_{k+1} + 1$ (if P_k is negative) or $2x_{k+1} + 1 - 2y_{k+1}$. Evaluation of terms $2x_{k+1}$ and $2y_{k+1}$ can also be done incrementally as: $2x_{k+1} = 2x_k + 2$, $2y_{k+1} = 2y_k - 2$. At the start position $(0, r)$, the initial decision parameter is obtained by evaluating the circle function at the start position $(x_0, y_0) = (0, r)$. So,

Algorithm

Step 1:

Input the radius (r) and the center of the circle (x_c, y_c).

Start from the first point:

- $x = 0$
- $y = r$

Step 2:

Calculate the initial decision parameter:

$$P = 1 - r$$

(This is the simplified formula used in practice.)

Step 3:

Repeat the following steps until $x \geq y$:

- If $P < 0$:
 - Increase x by 1.
 - Keep y the same.
 - Update:

$$P = P + 2x + 1$$

- Otherwise ($P \geq 0$):
 - Increase x by 1.
 - Decrease y by 1.
 - Update:

$$P = P + 2x + 1 - 2y$$

Step 4:

Use the circle's 8-way symmetry to find the remaining points in all eight octants.

Step 5:

Move every calculated point to the actual center of the circle:

- $X = x + x_c$
- $Y = y + y_c$

Plot these points on the screen.

Step 6:

Keep repeating Steps 3–5 until $x \geq y$. The circle is then complete.

Example: Given a circle radius $r = 10$, we demonstrate the midpoint circle algorithm by determining positions along the circle algorithm by determining positions along circle octant in the first quadrant from $x = 0$ to $x = y$.

The initial value of decision parameter is $P_0 = 1 - r = -9$. For the circle centered on the co-ordinates origin, the initial point is $(x_0, y_0) = (0, 10)$ and initial increment terms are: $2x_0 = 0$, $2y_0 = 20$.

Successive decision parameter values and positions along the circle path are calculated using the midpoint method as:

k	P_k	(x_{k+1}, y_{k+1})	$2x_{k+1}$	$2y_{k+1}$
0	-9	(1, 10)	2	20
1	-6	(2, 10)	4	20
2	-1	(3, 10)	6	20
3	6	(4, 9)	8	18
4	3	(5, 9)	10	18
5	8	(6, 8)	12	16
6	5	(7, 7)	14	14

ELLIPSE GENERATING ALGORITHM:

1. PROPERTIES OF THE ELLIPSE:

An ellipse is the set of points, such that the sum of the distances from two fixed position is the same for all positions (Foci) is the same for all points.

If the distances to the two foci from any point $P = (x, y)$ on the ellipse are labeled ' d_1 ' and ' d_2 ', then the general equation of ellipse stated as:

Properties of an Ellipse

1. Definition:

An ellipse is the set of all points such that the **sum of the distances from any point on the ellipse to two fixed points (called foci)** is constant. This is the fundamental property of an ellipse.

2. Constant Distance Property:

If F_1 and F_2 are the two foci and $P(x, y)$ is any point on the ellipse, then:

$$d_1 + d_2 = \text{Constant}$$

3. General Equation:

The general equation of an ellipse is:

$$Ax^2 + By^2 + Cxy + Dx + Ey + F = 0$$

4. Standard Equation:

For an ellipse centered at (x_c, y_c) with semi-major axis r_x and semi-minor axis r_y :

$$\frac{(x - x_c)^2}{r_x^2} + \frac{(y - y_c)^2}{r_y^2} = 1$$

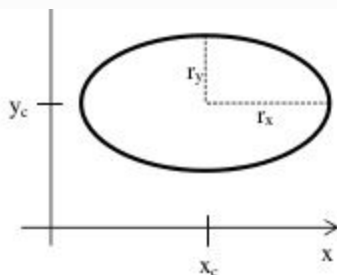
5. Parametric Equation:

The ellipse can be represented parametrically as:

$$x = x_c + r_x \cos \theta, \quad y = y_c + r_y \sin \theta$$

6. Symmetry:

An ellipse is symmetric about both the **x-axis and y-axis** (or about its major and minor axes). This symmetry helps reduce computation in ellipse drawing algorithms.



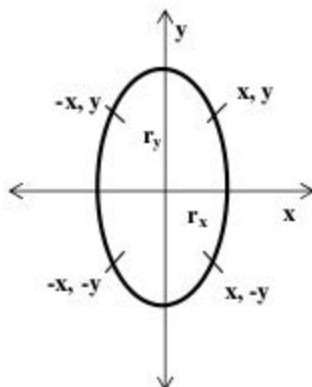
parameter ' r_y ' is the semi-minor axis. The equation of the ellipse can be written in terms of the ellipse center co-ordinates and parameters ' r_x ' and ' r_y ' as:

$$(x - x_c/r_x)^2 + (y - y_c/r_y)^2 = 1 \quad \text{--- (4)}$$

Using polar co-ordinates ' r ' and ' θ ', the parametric equations of the ellipse is:

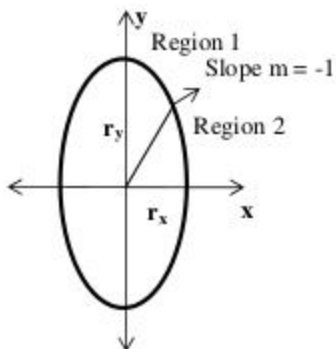
$$x = x_c + r_x \cos \theta \quad \text{and} \quad y = y_c + r_y \sin \theta \quad \dots (5)$$

Symmetric consideration can be used to further reduce computation.



2. MID-POINT ELLIPSE/BRESENHAM'S ELLIPSE GENERATING ALGORITHM:

This is applied throughout the 1st quadrant in 2 parts. The division of 1st quadrant is according to the slope of an ellipse with $r_x < r_y$. When we process this quadrant by taking unit steps in the x-direction, where slope of the curve has a magnitude less than 1 and taking unit steps in y-direction, when slope has the magnitude greater than 1.



For a sequential implementation of the midpoint algorithm, we take the start position at $(0, r_y)$ in clockwise order throughout the 1st quadrant. The ellipse function from equation (4) with $(x_c, y_c) = (0, 0)$ as:

$$F_{\text{ellipse}}(x, y) = r_y^2 x^2 + r_x^2 y^2 - r_x^2 r_y^2 \quad \dots (6)$$

It has 3 properties, at any point (x, y). That is:

$$F_{\text{ellipse}}(x, y) = \begin{cases} < 0, \text{ if } (x, y) \text{ is inside the ellipse boundary} \\ = 0, \text{ if } (x, y) \text{ is on the ellipse boundary} \quad \text{--- (7)} \\ > 0, \text{ if } (x, y) \text{ is outside the ellipse boundary} \end{cases}$$

Thus the ellipse function $F_{\text{ellipse}}(x, y)$ serves as the decision parameter in midpoint algorithm.

Starting at (0, ry), we take unit steps in the x-direction until we reach the boundary between region 1 and region 2. The slope is calculated from equation – (6) is:

$$m = dy/dx = -2r_y^2x/2r_x^2y \quad \text{--- (8)}$$

At the boundary between region 1 and region 2, $dy/dx = -1$ and $2r_y^2x = 2r_x^2y$. So we move to region 1, when $2r_y^2x \geq 2r_x^2y$.

In region 1, the midpoint between the two candidate pixels at sampling position $x_k + 1$ in the 1st region. Assuming position (x_k, y_k) has been selected at the previous step. We determine the next position along the ellipse path by evaluating the decision parameter at this midpoint is:

$$P_k^1 = F_{\text{ellipse}}(x_k+1, y_k - 1/2) = r_y^2(x_k+1)^2 + r_x^2(y_k - 1/2)^2 - r_x^2r_y^2 \quad \text{--- (9)}$$

If $P_k^1 < 0$, the midpoint is inside the ellipse and the pixel on scan line ' y_k ' is closer to the ellipse boundary, otherwise the midpoint is outside or on the ellipse boundary and we select the pixel on ' $y_k - 1$ '.

At the next sampling position ($x_{k+1} + 1 = x_k + 2$), the decision parameter for region 1 is evaluated as:

$$P_{k+1}^1 = F_{\text{ellipse}}(x_{k+1} + 1, y_{k+1} - 1/2) = r_y^2[(x_{k+1}) + 1]^2 + r_x^2(y_{k+1} - 1/2)^2 - r_x^2 r_y^2$$

$$\Rightarrow P_{k+1}^1 = P_k^1 + 2r_y^2(x_k + 1) + r_y^2 + r_x^2[(y_{k+1} - 1/2)^2 - (y_k - 1/2)^2] \dots (10),$$

Where, ' y_{k+1} ' is either ' y_k ' or ' $y_k - 1$ ', depending on the sign of P_k^1 .

Decision parameters are incremented by following amounts:

$$\text{Increment} = \begin{cases} 2r_y^2 x_k + 1 + r_y^2, & \text{if } P_k^1 < 0 \\ 2r_y^2 x_k + 1 + r_y^2 - 2r_x^2 y_k + 1, & \text{if } P_k^1 \geq 0 \end{cases}$$

At the initial position $(0, r_y)$, the two terms evaluated to

$$2r_y^2 x = 0 \dots (11)$$

$$2r_x^2 y = 2r_x^2 r_y \dots (12)$$

As ' x ' and ' y ' are incremented, updated values are obtained by adding $2r_y^2$ to equation – (11) and subtract $2r_x^2$ from equation – (12). The updated values are compared at each step and we move from region 1 to region 2, when $2r_y^2 x \geq 2r_x^2 y$ is satisfied.

In region 1, the initial value of decision parameter is obtained by evaluating the ellipse function at start position $(x_0, y_0) = (0, r_y)$.

$$\text{So, } P_0^1 = F_{\text{ellipse}}(1, r_y - 1/2) = r_y^2 - r_x^2(r_y - 1/2)^2 - r_x^2 r_y^2$$

$$\Rightarrow P_0^1 = r_y^2 - r_x^2 r_y - 1/4 r_x^2 \dots (13)$$

Over region 2, we sample at unit steps in negative y-direction and midpoint is now taken between horizontal pixels at each step. For this region 2, the decision parameter is evaluated as:

$$P_k^2 = F_{\text{ellipse}}(x_k + 1/2, y_k - 1) = r_y^2(x_k + 1/2)^2 + r_x^2(y_k - 1)^2 - r_x^2 r_y^2 \dots (14)$$

If $P_k^2 > 0$, the midpoint is outside the ellipse and we select the pixel at scan line ' x_k '. If $P_k^2 \leq 0$, the midpoint is inside or on the ellipse boundary and we select pixel position on ' $x_k + 1$ '.

To determine the relationship between successive decision parameters in region 2, we evaluate the ellipse function at the next sampling step: $y_{k+1} - 1 = y_k - 2$.

$$P_{k+1}^2 = F_{\text{ellipse}}(x_{k+1} + 1/2, y_{k+1} - 1) = r_y^2(x_{k+1} + 1/2)^2 + r_x^2[(y_k - 1) - 1]^2 - r_x^2 r_y^2$$

$$\Rightarrow P_{k+1}^2 = P_k^2 - 2r_x^2(y_k - 1)^2 + r_x^2 + r_y^2[(x_{k+1} + 1/2)^2 - (x_k + 1/2)^2] \dots (15),$$

Here, ' x_{k+1} ' set either ' x_k ' or ' x_{k+1} ', depending on the sign of P_k^2 .

When we enter region 2, the initial position (x_0, y_0) is taken as the last position selected in region 1 and the initial decision parameter in region 2 is:

$$P_0^2 = F_{\text{ellipse}}(x_0 + 1/2, y_0 - 1) = r_y^2(x_0 + 1/2)^2 - r_x^2(y_0 - 1)^2 - r_x^2 r_y^2 \dots (16)$$

To simplify the calculation of P_0^2 , we could select pixel position in counter clockwise order starting at $(r_x, 0)$ unit steps would then be taken in the positive y-direction upto the last position selected in region 1.

ALGORITHM:

STEP 1: Input r_x , r_y and ellipse center (x_c, y_c) and obtain the first point on an ellipse centered on the origin as: $(x_0, y_0) = (0, r_y)$.

STEP 2: Calculate the initial value of decision parameter in region 1 as:

$$P_0^1 = r_y^2 - r_x^2 r_y - 1/4 r_x^2$$

STEP 3: At each ' x_k ' position in region 1, starting at $k = 0$, perform the following test:

- i) If $P_k^1 < 0$, the next point along the ellipse centered on $(0, 0)$ is (x_{k+1}, y_k) and $P_{k+1}^1 = P_k^1 + 2r_y^2 x_{k+1} + r_y^2$, otherwise,
- ii) The next point along the circle is, (x_{k+1}, y_{k+1}) and
 $P_{k+1}^1 = P_k^1 + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_y^2$
With $2r_y^2 x_{k+1} = 2r_y^2 x_k + 2r_y^2$,
 $2r_x^2 y_{k+1} = 2r_x^2 y_k - 2r_x^2$ and continue until $2r_y^2 x \geq 2r_x^2 y$.

STEP 4: Calculate the initial value of the decision parameter in region 2, using the last point (x_0, y_0) . Calculated in region 1 as:

$$P_0^2 = r_y^2 (x_0 + 1/2)^2 - r_x^2 (y_0 - 1)^2 - r_x^2 r_y^2$$

STEP 5: At each ' y_k ' position in region 2, starting at $k = 0$, perform the following test:

- i) If $P_k^2 > 0$, the next point along the ellipse centered on $(0, 0)$ is (x_k, y_{k+1}) and $P_{k+1}^2 = P_k^2 - 2r_x^2 y_{k+1} + r_x^2$, otherwise,
- ii) The next point along the circle is, (x_{k+1}, y_{k+1}) and

$$P_{k+1}^2 = P_k^2 + 2r_y^2 x_{k+1} - 2r_x^2 y_{k+1} + r_x^2$$

Using the same incremental calculations for 'x' and 'y' as in region 1.

STEP 6: Determine symmetry points in the other 3 quadrants.

STEP 7: Move each calculated pixel position (x, y) onto the elliptical path centered on (x_c, y_c) and plot the co-ordinate values: x = x + x_c, y = y + y_c.

STEP 8: Repeat the steps for region 1, until $2r_y^2 x \geq 2r_x^2 y$.

Example: Given input ellipse parameter r_x = 8, r_y = 6. We illustrate the steps in the midpoint ellipse algorithm by determining raster positions along the ellipse path in the first quadrant. Initial values and increments for the decision parameter calculations as:

$$2r_y^2 = 0 \text{ (with increment } 2r_y^2 = 72) \text{ and}$$

$$2r_x^2 y = 2r_x^2 r_y \text{ (with increment } -2r_x^2 = -128)$$

For Region 1:

The initial point for the ellipse centered on the origin is (x₀, y₀) = (0, 6) and the initial decision parameter value is: $P_0^1 = r_y^2 - r_x^2 r_y - 1/4 r_x^2 = -332$.

Successive decision parameter values and positions along the ellipse path are calculated using the midpoint method as:

k	P_k^1	(x_{k+1}, y_{k+1})	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	-332	(1, 6)	72	768
1	-224	(2, 6)	144	768
2	-44	(3, 6)	216	768
3	208	(4, 5)	288	640
4	-108	(5, 5)	360	640
5	288	(6, 4)	432	512
6	244	(7, 3)	504	384

We now move out of region 1, since $2r_y^2 x > 2r_x^2 y$.

For Region 2:

The initial point is: $(x_0, y_0) = (7, 3)$ and the initial decision parameter value is: $P_0^2 = F_{\text{ellipse}}(7 + 1/2, 2) = -151$

The remaining positions along the ellipse path in the 1st quadrant are then calculated as:

k	P_k^2	(x_{k+1}, y_{k+1})	$2r_y^2 x_{k+1}$	$2r_x^2 y_{k+1}$
0	-151	(8, 2)	576	256
1	233	(8, 1)	576	128
2	745	(8, 0)	-	-

FILLED AREA PRIMITIVES:

A standard output primitives is a solid color or patterned polygon area. Other kinds of area primitives are sometimes available, but polygons are easier to process since they have linear boundaries.

Polygon Filling

Definition

Polygon Filling is the process of coloring or shading the interior of a closed polygon or defined region. The region can be classified into two types:

1. **Interior-Defined Region**
2. **Boundary-Defined Region**

Types of Polygon Filling

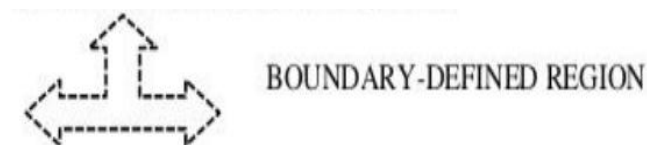
1. Interior-Defined Region

- The region is defined by its **interior pixels**.
- All pixels inside the region have the same color.
- The **Flood-Fill Algorithm** is used to fill this type of region.



2. Boundary-Defined Region

- The region is defined by its **boundary (outline) pixels**.
- The boundary has a unique color that separates the inside from the outside.
- The **Boundary-Fill Algorithm** is used to fill this type of region.



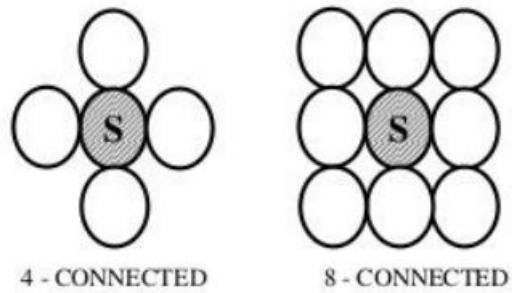
5. Boundary-Fill Algorithm

Definition

The **Boundary-Fill Algorithm** is a region-filling algorithm that fills an enclosed area with a specified color. It starts from a seed point inside the region and continues filling until it reaches the boundary color.

Steps

- Step 1:** Select a seed point inside the enclosed region.
- Step 2:** Check the color of the current pixel.
- Step 3:** If the pixel is neither the boundary color nor the fill color, change it to the fill color.
- Step 4:** Repeat the same process for all neighboring pixels (4-connected or 8-connected).
- Step 5:** Continue until the boundary color is reached and the entire region is filled.



6. Flood-Fill Algorithm

Definition

The Flood-Fill Algorithm is a region-filling algorithm that replaces all connected pixels of the same interior color with a new fill color. It starts from a seed point and continues until no connected pixel with the target color remains.

Steps

Step 1: Select a seed point inside the region.

Step 2: Check whether the current pixel has the target (old) color.

Step 3: If the pixel has the target color, replace it with the new fill color.

Step 4: Repeat the process for all neighboring pixels (4-connected or 8-connected).

Step 5: Continue until all connected pixels of the target color have been replaced.

